TaskFlow

Görev ve Proje Yönetim Sistemi REST API

Doküman	Proje tanıtımı ve tasarım çıktısı
Eğitim	Node.js ile Backend Programlama
Sürüm	1.0.0
Tarih	17 Ekim 2026

TaskFlow, bir yazılım ekibinin görevlerini, projelerini ve çalışanların sorumluluklarını tek bir yerden takip etmesini sağlayan bir REST API'dir.

Amaç

Proje, eğitim boyunca işlenen Node.js çalışma ortamı, Express ile sunucu geliştirme, routing, middleware, CRUD ve JSON veri yönetimi konularını gerçek bir iş problemini çözen tek bir sistemde birleştirir. Hedef yalnızca çalışan uçlar yazmak değil; tutarlı yanıt biçimi, açıklayıcı hata mesajları ve testlerle doğrulanmış, başka bir geliştiricinin kolayca entegre edebileceği bir API ortaya koymaktır.

Senaryo

Bir yazılım şirketi işleri e-posta ve tablolarla takip ediyor. Kimin hangi görevden sorumlu olduğu, hangi işlerin geciktiği ve projelerin ne kadar ilerlediği belirsiz.

TaskFlow ile ekip lideri görevi açıp bir çalışana atar, öncelik ve bitiş tarihi belirler. Çalışan görevi bekliyor, devam ediyor ve tamamlandı aşamalarından geçirir. Yönetici tamamlanma oranını, geciken işleri ve kişi başı iş yükünü raporlardan izler.

Sistem özellikleri

Özellik	Nasıl karşılanıyor	Kapsam
Görev yönetimi	Ekleme, listeleme, detay, tam ve kısmi güncelleme, silme	**Zorunlu**
İstek loglama	Her isteğin method, endpoint, zaman damgası, durum kodu ve süresi konsola ve dosyaya yazılır	**Zorunlu**
Postman testleri	7 klasörde 43 istek ve 198 otomatik doğrulama	**Zorunlu**
Gelişmiş sorgular	Durum ve önceliğe göre filtre, Türkçe karakter duyarsız arama, kişi bazlı liste, sayfalama, sıralama	İsteğe bağlı 01
Doğrulama	Alan bazlı mesajlarla 400 yanıtı; gövde ve sorgu parametreleri birlikte denetlenir	İsteğe bağlı 02
Raporlama	Tamamlanan ve bekleyen görev raporları, genel sistem özeti	İsteğe bağlı 03
Dokümantasyon	API dokümantasyonu, test senaryoları, kullanım kılavuzu	İsteğe bağlı 04
Çalışan ve proje yönetimi	Görevlerin bağlandığı çalışan ve proje kaynakları, ilişki kuralları	Senaryo gereği

Bir isteğin yolculuğu

Her istek aynı katmanlardan aynı sırayla geçer. Her katmanın tek bir görevi vardır ve bir alttaki katmanı parametre olarak alır.



Hata olursa Herhangi bir adımda oluşan hata merkezi `errorHandler` middleware'ine iletilir ve her zaman `{ "error": { "code", "message", "details" } }` biçiminde döner. Tanımsız yollar `notFoundHandler` ile 404 alır.

Veri modeli

Sistem üç kaynaktan oluşur. Veriler `data/` klasöründe her kaynak için ayrı bir JSON dosyasında tutulur. Görev, çalışana kullanıcı adıyla, projeye kimliğiyle bağlanır.

`Task.assignee → User.username` `Task.projectId → Project.id`

Görev — data/tasks.json

id	UUID, sunucu üretir
title	Zorunlu, 3-100 karakter
description	Zorunlu, en fazla 1000 karakter
status	pending, in-progress, completed (varsayılan pending)
priority	low, medium, high (varsayılan medium)
assignee	Kayıtlı çalışanın kullanıcı adı ya da null
projectId	Kayıtlı projenin kimliği ya da null
dueDate	YYYY-AA-GG biçiminde geçerli tarih ya da null
completedAt	Tamamlanınca sunucu atar, yeniden açılınca temizler
createdAt	Oluşturulma zamanı (ISO 8601)
updatedAt	Her güncellemede yenilenir

Çalışan — data/users.json

id	UUID
username	Benzersiz, a-z 0-9 . _ - ile 3-30 karakter, değiştirilemez
fullName	Zorunlu, 3-100 karakter
email	Benzersiz, geçerli e-posta
role	manager, developer, designer, tester
createdAt, updatedAt	ISO 8601

Proje — data/projects.json

id	UUID
name	Zorunlu, harf duyarsız benzersiz
description	En fazla 1000 karakter ya da null
status	planned, active, completed
createdAt, updatedAt	ISO 8601

Görev yaşam döngüsü

`pending` → `in-progress` → `completed`

Görev herhangi bir durumdan diğerine geçebilir. `completed` durumuna geçişte `completedAt` atanır ve görev tamamlanmış kaldıkça korunur. Bitiş tarihi bugünden önce olan tamamlanmamış görevler raporlarda gecikmiş sayılır.

İş kuralları

- Görev yalnızca sistemde kayıtlı bir çalışana ve projeye bağlanabilir; aksi halde alan bazlı mesajla 400 döner.
- Kullanıcı adı ve e-posta çalışanlar arasında, proje adı projeler arasında benzersizdir. Çakışma 409 Conflict ile bildirilir.
- Görevi olan çalışan veya proje silinemez; mesaj bağlı görev sayısını belirtir. Bu kural sahipsiz görev oluşmasını engeller.
- Kullanıcı adı görev ilişkisinin anahtarı olduğu için oluşturulduktan sonra değiştirilemez.
- Şemada tanımlı olmayan alanlar ve sorgu parametreleri sessizce yok sayılmaz, hata olarak bildirilir.
- `id`, `createdAt`, `updatedAt` ve `completedAt` alanlarını yalnızca sunucu yazar.

API tasarımı

Kaynak odaklı, durum kodlarını anlamına uygun kullanan bir REST tasarımı izlenmiştir: oluşturma 201 ve `Location` başlığı, silme 204, doğrulama hatası 400, bulunamayan kayıt 404, çakışma 409 döner. Endpoint yolları proje yönergesindeki örneklerle birebir aynıdır.

Method	Endpoint	Açıklama
Görevler		
POST	/tasks	Görev oluşturur
GET	/tasks	Listeler; status, priority, assignee, projectId, page, limit, sort, order
GET	/tasks/search?keyword=backend	Başlık ve açıklamada arar
GET	/tasks/assignee/musa	Bir çalışanın görevleri
GET	/tasks/:taskId	Görev detayı
PUT	/tasks/:taskId	Tam güncelleme, eksik isteğe bağlı alanlar varsayılana döner
PATCH	/tasks/:taskId	Kısmi güncelleme
DELETE	/tasks/:taskId	Görevi siler
Raporlar		
GET	/reports/completed	Tamamlanan sayısı, oran, öncelik dağılımı, ortalama süre
GET	/reports/pending	Bekleyen, devam eden, geciken ve atanmamış görevler
GET	/reports/summary	Toplamlar, dağılımlar, kişi başı iş yükü, proje ilerlemesi
Çalışanlar ve projeler		
GET POST	/users, /projects	Listeleme ve oluşturma
GET PUT	/users/:id, /projects/:id	Detay ve tam güncelleme
PATCH DELETE	/users/:id, /projects/:id	Kısmi güncelleme ve silme (bağlı görev varsa 409)
GET	/projects/:id/tasks	Projeye ait görevler
Sistem		
GET	/, /health	API bilgisi ve sağlık kontrolü

Yanıt biçimi

Başarılı liste yanıtı

```
{
  "data": [ { "id": "b77b798a-…", "title": "…" } ],
  "pagination": {
    "page": 1, "limit": 10, "totalItems": 12,
    "totalPages": 2, "hasNextPage": true,
    "hasPreviousPage": false
  }
}
```

Doğrulama hatası

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "İstek verileri doğrulanamadı.",
    "details": [{ "field": "priority",
      "location": "body", "message": "priority
      alanı şu değerlerden biri olmalıdır: …" }]
  }
}
```

Teknoloji ve kalite

127

otomatik test, `node:test` ve `supertest` ile, hepsi başarılı

43 / 198

Postman isteği ve doğrulaması, Newman ile hepsi başarılı

1

çalışma zamanı bağımlılığı (`express`); doğrulama, sorgu ve depolama kütüphanesiz